Session 8 Lab — Art Gallery Navigator

Task Description

In this lab, I built a console-based **Art Gallery Navigator** using Kotlin.The application displays artwork details and allows navigation using **Next** and **Previous** logic. This task helped me practice:

- Data classes
- List management
- Conditional logic
- Wrapping (circular navigation)
- Function design

Implementation

1. Data Class: Artwork

A data class is used to store structured information about each artwork.

```
data class Artwork(
    val title: String,
    val artist: String,
    val year: Int
)
```

2. Function: displayArtwork()

This function prints artwork details in a clean, formatted style.

```
fun displayArtwork(art: Artwork) {
    println("Viewing: ${art.title} (${art.artist}, ${art.year})")
}
```

3. Function: nextArtwork()

This function moves to the next artwork.
If the user is at the last artwork, it wraps back to index 0.

```
fun nextArtwork(currentIndex: Int, total: Int): Int {
    return if (currentIndex == total - 1) {
        0   // Wrap to first artwork
    } else {
```

```
        currentIndex + 1
    }
}
```

4. Function: previousArtwork()

This function moves to the previous artwork.
If the user is at the first artwork, it wraps to the last.

```
fun previousArtwork(currentIndex: Int, total: Int): Int {
    return if (currentIndex == 0) {
        total - 1   // Wrap to last artwork
    } else {
        currentIndex - 1
    }
}
```

5. Main Function (Simulation)

In main(), I:

- Created a list of artworks
- Started at the first artwork
- Simulated navigation using Next and Previous
- Displayed artwork details after each move

```
fun main() {

    val artworks = listOf(
        Artwork("Starry Night", "Vincent van Gogh", 1889),
        Artwork("Mona Lisa", "Leonardo da Vinci", 1503),
        Artwork("The Persistence of Memory", "Salvador Dalí", 1931),
        Artwork("The Scream", "Edvard Munch", 1893)
    )

    var currentIndex = 0

    // Display first artwork
    displayArtwork(artworks[currentIndex])

    // Simulate navigation
    println("-> Next Artwork")
```

```
    currentIndex = nextArtwork(currentIndex, artworks.size)
    displayArtwork(artworks[currentIndex])

    println("-> Next Artwork")
    currentIndex = nextArtwork(currentIndex, artworks.size)
    displayArtwork(artworks[currentIndex])

    println("-> Next Artwork")
    currentIndex = nextArtwork(currentIndex, artworks.size)
    displayArtwork(artworks[currentIndex])

    println("-> Next Artwork (Wrap Around)")
    currentIndex = nextArtwork(currentIndex, artworks.size)
    displayArtwork(artworks[currentIndex])

    println("-> Previous Artwork")
    currentIndex = previousArtwork(currentIndex, artworks.size)
    displayArtwork(artworks[currentIndex])
}
```

Sample Output

```
Viewing: Starry Night (Vincent van Gogh, 1889)
-> Next Artwork
Viewing: Mona Lisa (Leonardo da Vinci, 1503)
-> Next Artwork
Viewing: The Persistence of Memory (Salvador Dalí, 1931)
-> Next Artwork
Viewing: The Scream (Edvard Munch, 1893)
-> Next Artwork (Wrap Around)
Viewing: Starry Night (Vincent van Gogh, 1889)
-> Previous Artwork
Viewing: The Scream (Edvard Munch, 1893)
```

Concepts Practiced

Data Classes

Used to model real-world objects (artworks).

List Management

Stored multiple artworks in a list.

Conditional Logic

Used if/else to control navigation.

Wrapping Logic (Circular Navigation)

Ensured navigation loops correctly at boundaries.

Reflection

This lab strengthened my understanding of list indexing and conditional logic. Implementing wrapping logic helped me understand edge-case handling when working with collections. I also improved my ability to break problems into smaller reusable functions. This structure makes the program clean, readable, and scalable.